

QCMS

QCMS Enterprise OS

Azure Cloud Deployment Guide

Quality & Continuous Improvement Management System
Complete Technical Reference for Production Deployment on Microsoft Azure

Python 3.11+

Flask 3.1 / Gunicorn

PostgreSQL 17 + pgvector

Redis 7

Docker / Azure Container Apps

Azure Blob Storage

Azure Key Vault

GitHub Actions CI/CD

DOCUMENT TYPE

Deployment Guide

PREPARED

September 2026

CLASSIFICATION

Internal — Confidential

TARGET PLATFORM

Microsoft Azure

DEPLOYMENT MODEL

Azure Container Apps

PYTHON RUNTIME

Python 3.11+

QCMS Enterprise OS — Azure Cloud Deployment Guide

Document Type: Production Deployment Reference

Platform: Microsoft Azure

Prepared: September 2026

Classification: Internal — Confidential

Version: 1.0.0

Developer Handover — What the Developer Must Provide

This section is for the **Developer** handing over to the **Deployment Team**.

The deployment team cannot proceed without every item listed below.

Items marked  are already provided. Items marked  are pending.

1. GitHub Repository Access

Item	Status	Details
Repository URL	 Shared	github.com/IFQM-QCMS/imfq-QCMS
Branch to deploy	 Confirm	Deployment team needs to know: main or a release tag (e.g., v1.0.0)
Repository access	 Shared	Confirm the deployment team's GitHub account has read access to the repo

2. Environment Variables (.env File)

Item	Status	Details
.env file	 Shared	The developer shares the filled-in .env file with all actual production values

The deployment team needs the following values **filled in** (not placeholders) inside the [.env](#) file:

```
# — Core —————
FLASK_ENV=production
SECRET_KEY=<fill: generate using openssl rand -base64 48>
JWT_SECRET_KEY=<fill: generate separately from SECRET_KEY>
PORT=5000

# — Database —————
DATABASE_URL=<fill: postgresql://user:password@host:5432/dbname?sslmode=require>

# — Redis —————
REDIS_URL=<fill: rediss://default:key@host.redis.cache.windows.net:6380/0>
CELERY_BROKER_URL=<fill: same as REDIS_URL>
CELERY_RESULT_BACKEND=<fill: same host as REDIS_URL but /1>
REQUIRE_REDIS_SECURITY=true

# — Azure Storage —————
STORAGE_BACKEND=azure
AZURE_STORAGE_CONNECTION_STRING=<fill: from Azure Portal>
AZURE_STORAGE_CONTAINER_NAME=qcms-uploads
AZURE_STORAGE_BLOB_URL=<fill: https://<account>.blob.core.windows.net/qcms-uploads>

# — Email (Resend) —————
RESEND_API_KEY=<fill: re_XXXXXXXXXXXXXX>
RESEND_FROM_EMAIL=<fill: notifications@yourdomain.com>

# — Admin —————
SUPER_ADMIN_USERNAME=<fill: superadmin@yourcompany.com>
SUPER_ADMIN_PASSWORD=<fill: strong password>
DEFAULT_TEMP_PASSWORD=Welcome@123

# — Optional —————
GOOGLE_API_KEY=<optional: Google Maps API key>
```

3. External Service Credentials (API Keys)

The developer must provide access or API keys for these external services:

Service	What to Provide	Where Developer Gets It	Required?
Resend (Email)	API Key (<code>re_XXXX ...</code>) + verified sender email/domain	resend.com → Settings → API Keys	YES
Google Maps API	Google API Key	Google Cloud Console → APIs & Services → Credentials	Optional

i If the Resend sender domain is not yet verified, the developer must verify it in the Resend dashboard **before** handover. Emails will silently fail without domain verification.

4. Production Domain Name

Item	Status	What to Provide
Production domain/subdomain	☐ Confirm	e.g., <code>qcms.yourcompany.com</code> or <code>app.yourcompany.com</code>
DNS provider access	☐ Confirm	Deployment team needs access to add CNAME and TXT records for SSL certificate

5. Super Admin (First Login) Credentials

Item	What to Provide
SuperAdmin email	The email address for the first platform administrator account
SuperAdmin password	A strong initial password (the deployment team will be asked to change it immediately after first login)

🔑 These are set via `SUPER_ADMIN_USERNAME` and `SUPER_ADMIN_PASSWORD` in the `.env` file.
Change the password immediately after the first successful login.

6. Azure Subscription (If Not Managed by Deployment Team)

If the developer is providing Azure access rather than the deployment team creating their own:

Item	What to Provide
Azure Subscription ID	Found in Azure Portal → Subscriptions
Resource Group	Either create one or give the deployment team Contributor role on the subscription
Access level needed	Contributor on the Resource Group + Key Vault Administrator

If the deployment team manages Azure themselves, no Azure access is needed from the developer.

7. Quick Handover Summary

#	What	Provided By	Status
1	GitHub Repository URL + Branch	Developer	✅ Shared
2	Filled <code>.env</code> file with all production values	Developer	✅ Shared
3	Resend API Key + verified sender email	Developer	🟡 Confirm
4	Production domain name (e.g., <code>qcms.company.com</code>)	Developer / Business	🟡 Confirm
5	DNS provider access (to add CNAME/TXT for SSL)	Developer / IT	🟡 Confirm
6	SuperAdmin email + initial password	Developer	🟡 Confirm
7	Google Maps API key	Developer	🟡 Optional
8	Azure subscription access (if applicable)	Developer / IT	🟡 If needed

Once all items above are confirmed, the deployment team can proceed with **Section 5 onwards** in this document.

Table of Contents

1. Executive Summary
2. System Overview & Architecture
3. Technology Stack
4. Azure Services Required

5. Pre-Deployment Prerequisites
6. Azure Infrastructure Setup
7. Environment Variable Configuration
8. Docker Image Build & Push
9. Container App Deployment
10. Database Migrations
11. Custom Domain & SSL/TLS
12. Security & Compliance Hardening
13. Monitoring & Logging
14. CI/CD Pipeline — GitHub Actions
15. Post-Deployment Verification Checklist
16. Scaling & Performance Tuning
17. Backup & Disaster Recovery
18. Troubleshooting Reference
19. Cost Estimation
20. Appendix — Quick Reference Commands

1. Executive Summary

This document is the authoritative deployment reference for **QCMS Enterprise OS** (Quality & Continuous Improvement Management System) on **Microsoft Azure**. It covers every step required to:

- Provision all Azure cloud infrastructure
- Configure secrets and environment variables securely
- Build and push Docker images to Azure Container Registry
- Deploy all application services via Azure Container Apps
- Run database migrations and verify the deployment

QCMS Enterprise OS is a multi-tenant, enterprise-grade SaaS platform built for manufacturing plants, industrial enterprises, automotive OEMs, and quality institutions. It enforces the internationally recognized **8-Stage DMAIC / Quality Circle methodology**.

Deployment at a Glance

Category	Details
Target Platform	Microsoft Azure
Compute	Azure Container Apps (ACA)
Database	Azure Database for PostgreSQL Flexible Server (v17 + pgvector)
Cache / Broker	Azure Cache for Redis (v7, TLS)
File Storage	Azure Blob Storage (private container, SAS URLs)
Secrets	Azure Key Vault (Managed Identity)
Container Registry	Azure Container Registry (ACR)
Deployment Model	4 containerized microservices
Estimated Deploy Time	~70 minutes (first time)

2. System Overview & Architecture

2.1 Application Purpose

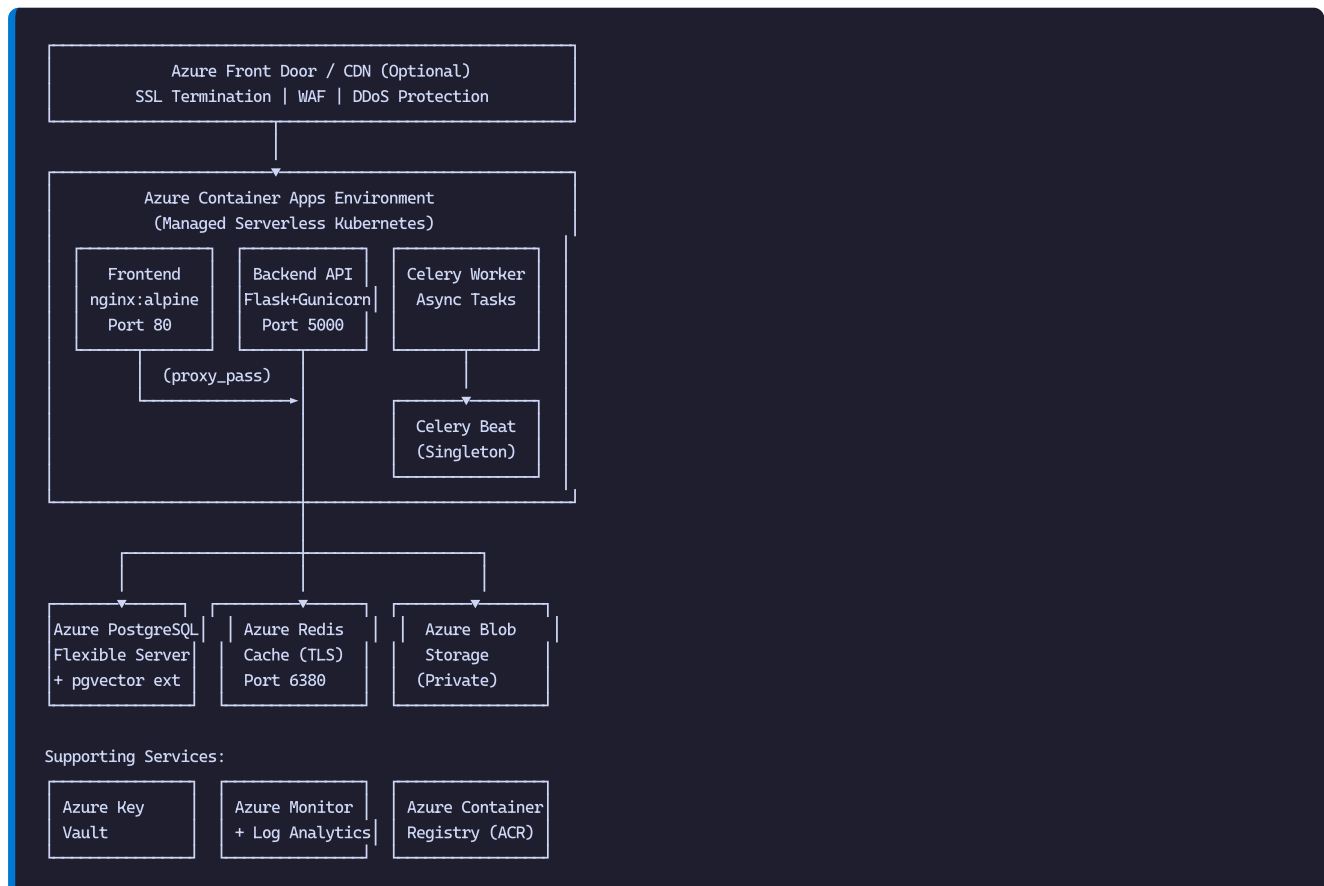
QCMS enforces an **8-stage DMAIC lifecycle**:

1. Problem Definition
2. Observation & Data Collection
3. Cause Identification
4. Root Cause Analysis (5-Why, Fishbone)
5. Countermeasure Planning
6. Implementation
7. Performance Verification
8. Standardization & Closure

Key Features:

- **7 QC Tools** — Ishikawa (Fishbone), Pareto 80/20, 5-Why, Scatter, SPC Charts, Histograms, Control Charts
- **AI-Powered RAG** — pgvector Retrieval-Augmented Generation recommends solutions from historical closed projects
- **Real-Time Collaboration** — Live presence rosters, heartbeat telemetry, concurrency collision locks
- **Automated PDF Reporting** — QC Storybooks and ISO 9001 compliance certificates via pdf2
- **144 Dynamic Feature Flags** — Granular module activation per organization subscription tier
- **Multilingual i18n** — 7 languages: English, Hindi, Kannada, Telugu, Tamil, Malayalam, Marathi
- **Multi-Tenant SaaS** — Complete DB and file storage isolation per tenant (org_id)
- **Session Heartbeat Termination** — Forced logout propagates to all active clients within 30 seconds
- **GPS Telemetry** — Login sessions capture IP and GPS coordinates for forensic audit logs

2.2 High-Level Azure Architecture



2.3 Microservice Components

Service	Docker Image	Port	Responsibilities	Replicas
Backend API	<code>qcms-backend</code>	5000	REST API, JWT auth, RBAC, ORM, PDF generation, RAG search	2–4
Celery Worker	<code>qcms-backend</code>	—	Async emails, background PDF gen, vector embedding indexing	1–2
Celery Beat	<code>qcms-backend</code>	—	Periodic tasks: session cleanup, presence TTL, health checks	1 (fixed)
Frontend	<code>qcms-frontend</code>	80	Nginx static server, SPA routing, API reverse proxy	2–5

 **Critical:** Celery Beat MUST always run as exactly **1 replica**. Multiple instances cause duplicate task scheduling and data corruption.

3. Technology Stack

3.1 Backend Stack (Python / Flask)

Component	Library / Tool	Version	Purpose
Runtime	Python	3.11+	Primary application language
Web Framework	Flask	>= 3.1.3	REST API server
ORM	SQLAlchemy + Flask-SQLAlchemy	>= 2.0.23 / 3.1.1	Database abstraction
JWT Authentication	Flask-JWT-Extended	>= 4.5.3	Token issuance & validation
Password Hashing	Flask-Bcrypt	>= 1.0.1	Bcrypt password hashing
CORS Policy	Flask-Cors	>= 4.0.2	Cross-origin resource sharing
DB Migrations	Flask-Migrate / Alembic	>= 4.0.5 / 1.13.1	Schema versioning
WSGI Server	Gunicorn (gthread)	>= 22.0.0	Production multi-worker server
Task Queue	Celery	>= 5.3.6	Async background processing
Cache / Broker	redis (Python client)	>= 5.0.1	Celery broker & cache
PDF Generation	fpdf2	>= 2.7.5	QC Storybooks & ISO certificates
Email	Resend API	>= 2.4.0	Transactional email delivery
Vector Search	pgvector	>= 0.2.4	RAG AI search embeddings
Numerical	NumPy	>= 1.24.0	Cosine similarity calculations
PostgreSQL Driver	psycopg2-binary + pg8000	>= 2.9.9 / 1.31.5	PostgreSQL adapters
HTTP Client	requests	>= 2.31.0	External API & webhook calls
Config Management	python-dotenv	>= 1.0.1	.env file loading

3.2 Frontend Stack (Vanilla JS / Nginx)

Component	Technology	Purpose
UI Framework	Vanilla ES6+ JavaScript (no React/Vue)	Lightweight SPA
Styling	Custom CSS3 — Glassmorphic Design System	Glass cards, CSS variables, dual themes
Utility CSS	Tailwind CSS	Responsive layout helpers
Icons	Lucide Icons (SVG)	Consistent icon set
Charts	Chart.js	Pareto, analytics, KPI charts
Web Server	Nginx:alpine	Static files, SPA routing, API proxy
Build Tool	Node.js + custom build.js	Minification, cache-busting
Linters	ESLint >= 10.9.1	JavaScript quality enforcement
CSS Minifier	clean-css >= 5.3.3	Production CSS optimization
JS Minifier	terser >= 5.50.0	JavaScript compression

3.3 Database & Storage

Service	Technology	Version	Role
Primary Database	PostgreSQL + pgvector	17.x (Azure)	35+ relational entities: Users, Orgs, Projects, Stages, SOPs, Audit Logs, Vectors
Cache & Broker	Redis	7.x (Azure Cache)	Session cache, rate limiting, Celery task queue broker & result backend
File Storage	Azure Blob Storage	Latest	Private container: uploads, PDFs, avatars served via 15-min SAS URLs
Secrets Management	Azure Key Vault	Latest	DB passwords, JWT secrets, API keys, storage connection strings

3.4 Infrastructure & DevOps

Component	Technology	Purpose
Containerization	Docker (python:3.11-slim / nginx:alpine)	Backend & Frontend container images
Orchestration	Azure Container Apps (ACA)	Serverless container hosting with auto-scaling
Container Registry	Azure Container Registry (ACR)	Private Docker image repository
CI/CD	GitHub Actions	Automated build, test, push & deploy
CDN / Edge	Azure Front Door / Custom DNS	SSL termination, WAF, global routing
Monitoring	Azure Monitor + Log Analytics	Telemetry, metrics, alerts
Networking	Azure Virtual Network	Private VNet for all backend services

4. Azure Services Required

Note: You need an active Azure subscription with **Contributor** role on the target Resource Group. For Key Vault, you additionally need **Key Vault Administrator** role.

#	Azure Service	Recommended SKU	Purpose	Est. Monthly
1	Azure Container Apps Environment	Consumption / Dedicated	Hosts all 4 containers with auto-scaling	~\$40–\$80
2	Azure Container Registry (ACR)	Basic / Standard	Private Docker image storage	~\$5–\$20
3	Azure DB for PostgreSQL Flexible Server	Standard_B2ms (2 vCores, 8 GB)	Primary relational DB + pgvector	~\$50–\$200
4	Azure Cache for Redis	C1 Basic → C3 Standard (prod)	Caching layer & Celery message broker	~\$30–\$100
5	Azure Blob Storage Account	Standard LRS → GRS (prod)	File uploads, PDFs, avatars (private)	~\$5–\$30
6	Azure Key Vault	Standard	Secrets, API keys, connection strings	~\$5
7	Azure Virtual Network	Standard	Private networking for backend services	~\$5–\$20
8	Azure Monitor / Log Analytics	Pay-as-you-go	Centralized logs, metrics, alerts	~\$10–\$30
9	Azure Front Door + WAF	Standard (Optional)	Global CDN, SSL, DDoS protection	~\$35+

5. Pre-Deployment Prerequisites

5.1 Local Machine Requirements

Tool	Version	Download
Azure CLI (az)	v2.60+	https://docs.microsoft.com/cli/azure
Docker Desktop	v25+	https://www.docker.com/products/docker-desktop
Git	v2.40+	https://git-scm.com
Node.js	v18+ LTS	https://nodejs.org
Python	3.11+	https://www.python.org (optional — for local migrations)

5.2 Required Credentials & API Keys

Credential	Where to Obtain	Required?
Azure Subscription ID	Azure Portal → Subscriptions	YES
Resend API Key	resend.com → Settings → API Keys	YES
Google API Key	Google Cloud Console → APIs & Services	OPTIONAL
GitHub Personal Access Token	GitHub → Settings → Developer Settings → PAT	YES (CI/CD)
Super Admin Email & Password	Define securely before first deployment	YES

5.3 Azure CLI Login

```
# Login to Azure (opens browser)
az login

# Set the active subscription
az account set --subscription "<YOUR_SUBSCRIPTION_ID>"

# Verify the correct subscription is active
az account show
```

6. Azure Infrastructure Setup

 **Important:** Run the commands in order. Each step depends on resources created in the previous step. Store all generated passwords and connection strings in Azure Key Vault immediately.

6.1 Define Shell Variables (Run Once)

```
LOCATION="eastus"
RG="qcms-production-rg"
ACR_NAME="qcmsregistry$(openssl rand -hex 3)" # Must be globally unique
ACA_ENV="qcms-aca-env"
BACKEND_APP="qcms-backend"
FRONTEND_APP="qcms-frontend"
CELERY_WORKER_APP="qcms-celery-worker"
CELERY_BEAT_APP="qcms-celery-beat"
PG_SERVER="qcms-postgres-server"
REDIS_NAME="qcms-redis-cache"
STORAGE_ACCOUNT="qcmsstorage$(openssl rand -hex 3)" # Must be globally unique
STORAGE_CONTAINER="qcms-uploads"
KEYVAULT_NAME="qcms-kv-$(openssl rand -hex 3)"
PG_ADMIN_USER="qcmsadmin"
PG_ADMIN_PASSWORD="$(openssl rand -base64 24)QcMs@1" # SAVE THIS!
PG_DB_NAME="qcms_db"

# Create Resource Group
az group create --name $RG --location $LOCATION \
  --tags Project=QCMS Environment=Production
```

6.2 Azure PostgreSQL Flexible Server

 **pgvector is required** for the AI RAG search feature. Must be enabled after server creation.

```
# Create the PostgreSQL Flexible Server
az postgres flexible-server create \
  --resource-group $RG \
  --name $PG_SERVER \
  --location $LOCATION \
  --admin-user $PG_ADMIN_USER \
  --admin-password "$PG_ADMIN_PASSWORD" \
  --sku-name Standard_B2ms \
  --tier Burstable \
  --version 17 \
  --storage-size 32 \
  --backup-retention 7

# Create the application database
az postgres flexible-server db create \
  --resource-group $RG \
  --server-name $PG_SERVER \
  --database-name $PG_DB_NAME

# Enable pgvector extension
az postgres flexible-server parameter set \
  --resource-group $RG \
  --server-name $PG_SERVER \
  --name azure.extensions \
  --value VECTOR

# Allow Azure services to connect
az postgres flexible-server firewall-rule create \
  --resource-group $RG \
  --name $PG_SERVER \
  --rule-name AllowAzureServices \
  --start-ip-address 0.0.0.0 \
  --end-ip-address 0.0.0.0

# Construct the DATABASE_URL
echo "DATABASE_URL=postgresql://${PG_ADMIN_USER}:${PG_ADMIN_PASSWORD}@${PG_SERVER}.postgres.database.azure.com:5432/${PG_DB_NAME}?sslmode=require"
```

6.3 Azure Cache for Redis

 **Critical:** Azure Cache for Redis uses **TLS on port 6380**. You MUST use `rediss://` (double `s`) — NOT `redis://`. Using the wrong protocol causes an immediate connection refused error.

```
# Create Redis Cache
az redis create \
  --resource-group $RG \
  --name $REDIS_NAME \
  --location $LOCATION \
  --sku Standard \
  --vm-size C1 \
  --redis-version 7

# Retrieve connection details
REDIS_HOST=$(az redis show -g $RG -n $REDIS_NAME --query 'hostName' -o tsv)
REDIS_KEY=$(az redis list-keys -g $RG -n $REDIS_NAME --query 'primaryKey' -o tsv)

# Construct REDIS_URL (note: rediss:// with TLS, port 6380)
echo "REDIS_URL=rediss://default:${REDIS_KEY}@${REDIS_HOST}:6380/0"
echo "CELERY_BROKER_URL=rediss://default:${REDIS_KEY}@${REDIS_HOST}:6380/0"
echo "CELERY_RESULT_BACKEND=rediss://default:${REDIS_KEY}@${REDIS_HOST}:6380/1"
```

6.4 Azure Blob Storage (Private Container)

```
# Create Storage Account
az storage account create \
  --name $STORAGE_ACCOUNT \
  --resource-group $RG \
  --location $LOCATION \
  --sku Standard_LRS \
  --kind StorageV2 \
  --allow-blob-public-access false \
  --min-tls-version TLS1_2

# Create private blob container
az storage container create \
  --name $STORAGE_CONTAINER \
  --account-name $STORAGE_ACCOUNT \
  --public-access off

# Get connection string
AZURE_STORAGE_CONN=$(az storage account show-connection-string \
  --resource-group $RG \
  --name $STORAGE_ACCOUNT \
  --query 'connectionString' -o tsv)

echo "AZURE_STORAGE_CONNECTION_STRING=${AZURE_STORAGE_CONN}"
```

6.5 Azure Container Registry (ACR)

```
# Create Container Registry
az acr create \
  --resource-group $RG \
  --name $ACR_NAME \
  --sku Standard \
  --admin-enabled true \
  --location $LOCATION

# Get login server
ACR_LOGIN_SERVER=$(az acr show --name $ACR_NAME --query 'loginServer' -o tsv)

# Login to ACR
az acr login --name $ACR_NAME

echo "ACR Login Server: ${ACR_LOGIN_SERVER}"
```

6.6 Azure Container Apps Environment

```
# Create Log Analytics Workspace
WORKSPACE_ID=$(az monitor log-analytics workspace create \
  --resource-group $RG \
  --workspace-name qcms-log-workspace \
  --location $LOCATION \
  --query 'customerId' -o tsv)

WORKSPACE_KEY=$(az monitor log-analytics workspace get-shared-keys \
  --resource-group $RG \
  --workspace-name qcms-log-workspace \
  --query 'primarySharedKey' -o tsv)

# Create Container Apps Environment
az containerapp env create \
  --name $ACA_ENV \
  --resource-group $RG \
  --location $LOCATION \
  --logs-workspace-id $WORKSPACE_ID \
  --logs-workspace-key $WORKSPACE_KEY
```

6.7 Azure Key Vault (Secrets Storage)

```
# Create Key Vault
az keyvault create \
  --name $KEYVAULT_NAME \
  --resource-group $RG \
  --location $LOCATION \
  --sku standard

# Store secrets (repeat for each variable)
az keyvault secret set \
  --vault-name $KEYVAULT_NAME \
  --name "DATABASE-URL" \
  --value "postgresql://$PG_ADMIN_USER:$PG_ADMIN_PASSWORD@$PG_SERVER.postgres.database.azure.com:5432/$PG_DB_NAME?sslmode=require"

az keyvault secret set \
  --vault-name $KEYVAULT_NAME \
  --name "SECRET-KEY" \
  --value "$(openssl rand -base64 48)"

az keyvault secret set \
  --vault-name $KEYVAULT_NAME \
  --name "JWT-SECRET-KEY" \
  --value "$(openssl rand -base64 48)"

az keyvault secret set \
  --vault-name $KEYVAULT_NAME \
  --name "REDIS-URL" \
  --value "redis://default:${REDIS_KEY}@${REDIS_HOST}:6380/0"

az keyvault secret set \
  --vault-name $KEYVAULT_NAME \
  --name "AZURE-STORAGE-CONN" \
  --value "${AZURE_STORAGE_CONN}"

az keyvault secret set \
  --vault-name $KEYVAULT_NAME \
  --name "RESEND-API-KEY" \
  --value "re_your_production_key_here"
```

7. Environment Variable Configuration

 **Security:** Generate strong secrets using `openssl rand -base64 48`. These must be at least 48 characters. **Never reuse development secrets in production.** All secrets must be stored in Azure Key Vault.

7.1 Core Application Variables

Variable	Required	Value / Description
FLASK_ENV	YES	<code>production</code> — enables production mode, disables debug
SECRET_KEY	YES	Flask session secret. Generate: <code>openssl rand -base64 48</code>
JWT_SECRET_KEY	YES	JWT signing key. Generate separately from <code>SECRET_KEY</code>
PORT	YES	<code>5000</code> — backend API listening port

7.2 Database

Variable	Required	Value / Description
DATABASE_URL	YES	<code>postgresql://qcmsadmin:<pw>@<server>.postgres.database.azure.com:5432/qcms_db?sslmode=require</code>

7.3 Redis & Celery

Variable	Required	Value / Description
REDIS_URL	YES	<code>rediss://default:<key>@<host>.redis.cache.windows.net:6380/0</code> (must be <code>rediss://</code>)
CELERY_BROKER_URL	YES	Same as <code>REDIS_URL</code> (database index <code>/0</code>)
CELERY_RESULT_BACKEND	YES	Same host, database index <code>/1</code>
REQUIRE_REDIS_SECURITY	YES	<code>true</code> — fail-closed: denies all requests if Redis drops

7.4 Azure Storage

Variable	Required	Value / Description
STORAGE_BACKEND	YES	<code>azure</code> — activates Azure Blob Storage provider
AZURE_STORAGE_CONNECTION_STRING	YES	Full connection string from Azure Portal
AZURE_STORAGE_CONTAINER_NAME	YES	<code>qcms-uploads</code>
AZURE_STORAGE_BLOB_URL	YES	<code>https://<account>.blob.core.windows.net/qcms-uploads</code>
MAX_CONTENT_LENGTH	OPTIONAL	<code>16777216</code> (16 MB upload limit)

7.5 Email & Notifications

Variable	Required	Value / Description
RESEND_API_KEY	YES	Resend.com API key (starts with <code>re_</code>)
RESEND_FROM_EMAIL	YES	<code>notifications@yourdomain.com</code> — must be verified in Resend

7.6 Administrator Setup

Variable	Required	Value / Description
SUPER_ADMIN_USERNAME	YES	Platform SuperAdmin email (e.g., <code>superadmin@yourco.com</code>)
SUPER_ADMIN_PASSWORD	YES	Strong password — change immediately after first login!
DEFAULT_TEMP_PASSWORD	OPTIONAL	<code>Welcome@123</code> — temp password for new user invitations
GOOGLE_API_KEY	OPTIONAL	Google Maps API key for GPS location telemetry

7.7 Gunicorn Performance Tuning

Variable	Default	Description
GUNICORN_WORKERS	Auto 2–4	Number of worker processes (based on CPU cores)
GUNICORN_THREADS	4	Threads per worker (gthread mode)
GUNICORN_TIMEOUT	60	Request timeout in seconds
GUNICORN_MAX_REQUESTS	1000	Recycle worker after N requests (prevents memory leaks)
LOG_LEVEL	info	Log level: <code>debug</code> / <code>info</code> / <code>warning</code> / <code>error</code>

8. Docker Image Build & Push

8.1 Build Frontend Assets (Node.js)

```
# Navigate to frontend directory
cd frontend/

# Install dependencies
npm install

# Build - minifies JS/CSS, generates cache-busted asset filenames
npm run build

# Verify output
ls -la assets/dist/
```

8.2 Build & Push Backend Image

```
# Tag with git SHA for traceability
QCMS_TAG=$(git rev-parse --short HEAD)

# Build backend image (uses python:3.11-slim base)
docker build \
  -t ${ACR_LOGIN_SERVER}/qcms-backend:${QCMS_TAG} \
  -t ${ACR_LOGIN_SERVER}/qcms-backend:latest \
  -f backend/Dockerfile ./backend

# Push to ACR
docker push ${ACR_LOGIN_SERVER}/qcms-backend:${QCMS_TAG}
docker push ${ACR_LOGIN_SERVER}/qcms-backend:latest

echo "Backend image pushed: ${ACR_LOGIN_SERVER}/qcms-backend:${QCMS_TAG}"
```

8.3 Build & Push Frontend Image

```
# Build frontend image (uses nginx:alpine base)
docker build \
  -t ${ACR_LOGIN_SERVER}/qcms-frontend:${QCMS_TAG} \
  -t ${ACR_LOGIN_SERVER}/qcms-frontend:latest \
  -f frontend/Dockerfile ./frontend

# Push to ACR
docker push ${ACR_LOGIN_SERVER}/qcms-frontend:${QCMS_TAG}
docker push ${ACR_LOGIN_SERVER}/qcms-frontend:latest
```

 **Alternative:** Build directly in Azure without local Docker:

```
bash az acr build --registry $ACR_NAME \ --image qcms-backend:latest \ --file backend/Dockerfile ./backend
```

9. Container App Deployment

All 4 services are deployed to the same Container Apps Environment and communicate via internal DNS.

9.1 Backend API — Internal Ingress

```
az containerapp create \
  --name $BACKEND_APP \
  --resource-group $RG \
  --environment $ACA_ENV \
  --image ${ACR_LOGIN_SERVER}/qcms-backend:latest \
  --target-port 5000 \
  --ingress internal \
  --min-replicas 2 \
  --max-replicas 4 \
  --cpu 1.0 \
  --memory 2Gi \
  --env-vars \
    FLASK_ENV=production \
    PORT=5000 \
    STORAGE_BACKEND=azure \
    REQUIRE_REDIS_SECURITY=true \
    DATABASE_URL=secretref:database-url \
    SECRET_KEY=secretref:secret-key \
    JWT_SECRET_KEY=secretref:jwt-secret-key \
    REDIS_URL=secretref:redis-url \
    CELERY_BROKER_URL=secretref:celery-broker-url \
    CELERY_RESULT_BACKEND=secretref:celery-result-backend \
    AZURE_STORAGE_CONNECTION_STRING=secretref:azure-storage-conn \
    RESEND_API_KEY=secretref:resend-api-key \
  --secrets \
    database-url=keyvaultref:https://${KEYVAULT_NAME}.vault.azure.net/secrets/DATABASE-URL \
    secret-key=keyvaultref:https://${KEYVAULT_NAME}.vault.azure.net/secrets/SECRET-KEY \
    jwt-secret-key=keyvaultref:https://${KEYVAULT_NAME}.vault.azure.net/secrets/JWT-SECRET-KEY \
    redis-url=keyvaultref:https://${KEYVAULT_NAME}.vault.azure.net/secrets/REDIS-URL \
    celery-broker-url=keyvaultref:https://${KEYVAULT_NAME}.vault.azure.net/secrets/REDIS-URL \
    celery-result-backend=keyvaultref:https://${KEYVAULT_NAME}.vault.azure.net/secrets/REDIS-URL \
    azure-storage-conn=keyvaultref:https://${KEYVAULT_NAME}.vault.azure.net/secrets/AZURE-STORAGE-CONN \
    resend-api-key=keyvaultref:https://${KEYVAULT_NAME}.vault.azure.net/secrets/RESEND-API-KEY
```

9.2 Celery Worker

```
az containerapp create \
  --name $CELERY_WORKER_APP \
  --resource-group $RG \
  --environment $ACA_ENV \
  --image ${ACR_LOGIN_SERVER}/qcms-backend:latest \
  --ingress disabled \
  --min-replicas 1 \
  --max-replicas 2 \
  --cpu 0.5 \
  --memory 1Gi \
  --command "celery" "-A" "celery_worker.celery" "worker" \
    "--loglevel=info" "--concurrency=2" \
  --env-vars \
    FLASK_ENV=production \
    DATABASE_URL=secretref:database-url \
    CELERY_BROKER_URL=secretref:celery-broker-url \
    CELERY_RESULT_BACKEND=secretref:celery-result-backend \
    AZURE_STORAGE_CONNECTION_STRING=secretref:azure-storage-conn \
    RESEND_API_KEY=secretref:resend-api-key \
  --secrets \
    database-url=keyvaultref:https://${KEYVAULT_NAME}.vault.azure.net/secrets/DATABASE-URL \
    celery-broker-url=keyvaultref:https://${KEYVAULT_NAME}.vault.azure.net/secrets/REDIS-URL \
    celery-result-backend=keyvaultref:https://${KEYVAULT_NAME}.vault.azure.net/secrets/REDIS-URL \
    azure-storage-conn=keyvaultref:https://${KEYVAULT_NAME}.vault.azure.net/secrets/AZURE-STORAGE-CONN \
    resend-api-key=keyvaultref:https://${KEYVAULT_NAME}.vault.azure.net/secrets/RESEND-API-KEY
```

9.3 Celery Beat Scheduler — SINGLETON ⚠️

 **Critical:** `--min-replicas 1 --max-replicas 1` is **mandatory**. Multiple Celery Beat instances cause duplicate scheduled tasks and data corruption. Never increase max replicas for this service.


```
az containerapp create \
  --name $CELERY_BEAT_APP \
  --resource-group $RG \
  --environment $ACA_ENV \
  --image ${ACR_LOGIN_SERVER}/qcms-backend:latest \
  --ingress disabled \
  --min-replicas 1 \
  --max-replicas 1 \
  --cpu 0.25 \
  --memory 0.5Gi \
  --command "celery" "-A" "celery_worker.celery" "beat" "--loglevel=info" \
  --env-vars \
    FLASK_ENV=production \
    DATABASE_URL=secretref:database-url \
    CELERY_BROKER_URL=secretref:celery-broker-url \
  --secrets \
    database-url=keyvaultref:https://${KEYVAULT_NAME}.vault.azure.net/secrets/DATABASE-URL \
    celery-broker-url=keyvaultref:https://${KEYVAULT_NAME}.vault.azure.net/secrets/REDIS-URL
```

9.4 Frontend — External Public Ingress

```
az containerapp create \
  --name $FRONTEND_APP \
  --resource-group $RG \
  --environment $ACA_ENV \
  --image ${ACR_LOGIN_SERVER}/qcms-frontend:latest \
  --target-port 80 \
  --ingress external \
  --min-replicas 2 \
  --max-replicas 5 \
  --cpu 0.5 \
  --memory 1Gi

# Get the public URL
FRONTEND_URL=$(az containerapp show \
  --name $FRONTEND_APP \
  --resource-group $RG \
  --query 'properties.configuration.ingress.fqdn' -o tsv)

echo "QCMS is live at: https://${FRONTEND_URL}"
```

10. Database Migrations

 **Run BEFORE serving user traffic.** This creates all 35+ database tables, indexes, and enables the pgvector extension. Missing migrations causes immediate application crashes.

```
# Step 1: Create a one-time Container App Job for migrations
az containerapp job create \
  --name qcms-migration-job \
  --resource-group $RG \
  --environment $ACA_ENV \
  --trigger-type Manual \
  --replica-timeout 300 \
  --replica-retry-limit 1 \
  --image ${ACR_LOGIN_SERVER}/qcms-backend:latest \
  --command "flask" "db" "upgrade" \
  --env-vars \
    FLASK_APP=run.py \
    FLASK_ENV=production \
    DATABASE_URL=secretref:database-url \
  --secrets \
    database-url=keyvaultref:https://${KEYVAULT_NAME}.vault.azure.net/secrets/DATABASE-URL

# Step 2: Start the migration job
az containerapp job start \
  --name qcms-migration-job \
  --resource-group $RG

# Step 3: Monitor job status
az containerapp job execution list \
  --name qcms-migration-job \
  --resource-group $RG \
  --output table

# Step 4: Enable pgvector extension in the database
PGPASSWORD="${PG_ADMIN_PASSWORD}" psql \
  -h "${PG_SERVER}.postgres.database.azure.com" \
  -U "${PG_ADMIN_USER}" \
  -d "${PG_DB_NAME}" \
  -c "CREATE EXTENSION IF NOT EXISTS vector;"
```

11. Custom Domain & SSL/TLS

Azure Container Apps provides **free managed TLS certificates** (Let's Encrypt) for custom domains.

Step 1 — Add Custom Domain:

```
az containerapp hostname add \
  --name $FRONTEND_APP \
  --resource-group $RG \
  --hostname qcms.yourdomain.com
```

Step 2 — Create DNS Records (in your DNS provider):

CNAME	qcms	→	<frontend-fqdn>.azurecontainerapps.io
TXT	asuid.qcms	→	<domain-verification-id> (provided by Azure)

Step 3 — Bind Managed Certificate:

```
az containerapp hostname bind \
  --name $FRONTEND_APP \
  --resource-group $RG \
  --hostname qcms.yourdomain.com \
  --environment $ACA_ENV \
  --validation-method CNAME
```

Azure automatically provisions a Let's Encrypt TLS certificate. HTTPS is enforced automatically.

Step 4 — Update Nginx proxy_pass:

Before rebuilding the frontend Docker image, update `frontend/nginx.conf` to point `proxy_pass` to the internal backend ACA FQDN (e.g., `http://qcms-backend.internal.azurecontainerapps.io:5000`).

12. Security & Compliance Hardening

12.1 Network Security Controls

Control	Configuration	Status
Backend Ingress	Set to INTERNAL — not publicly accessible. Only Nginx frontend can reach it.	Required
Redis TLS	Use <code>rediss://</code> (double <code>s</code>) on port 6380 . No plain-text connections.	Enforced
PostgreSQL SSL	<code>sslmode=require</code> in connection string. Firewall restricted to Azure services.	Enforced
Blob Storage	Private container — no public blob access. Files served via 15-min SAS URLs only.	Enforced
Secrets	All credentials in Azure Key Vault. No secrets in Docker images or environment vars directly.	Required

12.2 Application Security Controls

- **JWT Authentication** — All API endpoints (except `/health` , `/login` , `/register`) require valid JWT bearer token
- **RBAC — 7 Canonical Roles** — `SuperAdmin` , `Admin` , `CEO` , `Reviewer` , `Facilitator` , `Team Leader` , `Team Member` ; enforced via decorators on every Flask route
- **Multi-Tenant Isolation** — All DB queries scoped by `org_id` — cross-tenant data access is architecturally impossible
- **ActionLock / Idempotency** — Write operations deduplicated via `Idempotency-Key` headers (prevents double-submit)
- `REQUIRE_REDIS_SECURITY=true` — App fails **closed** (denies all requests) if Redis connection drops
- **Non-root Containers** — Backend runs as `USER qcms` ; Frontend runs as `USER nginx` — never as root
- **Bcrypt Password Hashing** — All user passwords hashed with bcrypt (never stored in plain text)
- **Session Heartbeat** — Forced logout propagates to all active clients within 30 seconds
- **GPS Telemetry** — Login sessions capture client IP and GPS coordinates for forensic audit logs

12.3 CORS Configuration for Production

```
# In backend/app/__init__.py
CORS(app, origins=[
    "https://qcms.yourdomain.com",
    "https://www.yourdomain.com"
], supports_credentials=True)

# NEVER use origins=['*'] in production
```

12.4 Managed Identity for Key Vault

```
# Assign system-assigned managed identity to all backend services
for app in $BACKEND_APP $CELERY_WORKER_APP $CELERY_BEAT_APP; do
  az containerapp identity assign \
    --name $app \
    --resource-group $RG \
    --system-assigned
done

# Get backend principal ID
BACKEND_PRINCIPAL=$(az containerapp show \
  --name $BACKEND_APP \
  --resource-group $RG \
  --query 'identity.principalId' -o tsv)

# Grant Key Vault Secret Reader permissions
az keyvault set-policy \
  --name $KEYVAULT_NAME \
  --object-id $BACKEND_PRINCIPAL \
  --secret-permissions get list
```

13. Monitoring & Logging

13.1 Health Check Endpoints

Service	Endpoint	Expected Response
Backend API	GET /api/health	HTTP 200 with JSON status
Backend Auth	GET /api/auth/maintenance-status	HTTP 200 — used by Docker health probe
Frontend	GET /	HTTP 200 — Nginx serving SPA index.html

13.2 Stream Container Logs

```
# Stream backend logs live
az containerapp logs show \
  --name qcms-backend \
  --resource-group $RG \
  --follow

# Stream Celery worker logs
az containerapp logs show \
  --name qcms-celery-worker \
  --resource-group $RG \
  --follow

# Query historical logs via Log Analytics KQL
az monitor log-analytics query \
  --workspace $WORKSPACE_ID \
  --analytics-query "
  ContainerAppConsoleLogs_CL
  | where ContainerAppName_s == 'qcms-backend'
  | order by TimeGenerated desc
  | take 100
"
```

13.3 Recommended Azure Monitor Alerts

Alert Name	Condition	Recommended Action
Backend Unhealthy	Health probe failure > 3 consecutive	Email + PagerDuty notification
High CPU	CPU > 80% for 5 minutes	Auto scale out replicas
High Memory	Memory > 85% of limit	Investigate memory leak in backend
Redis Unavailable	Connection errors spike	IMMEDIATE — fail-close activates
PostgreSQL Connections	Connections > 80% of <code>max_connections</code>	Review SQLAlchemy pool settings
HTTP 5xx Rate	> 1% of requests return 5xx	Investigation alert

14. CI/CD Pipeline — GitHub Actions

Create `.github/workflows/azure-deploy.yml` in the repository root:

```

name: QCMS Azure Production Deployment

on:
  push:
    branches: [main]
  workflow_dispatch:

env:
  RG: qcms-production-rg
  ACA_ENV: qcms-aca-env

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout Code
        uses: actions/checkout@v4

      - name: Set Image Tag
        run: echo "QCMS_TAG=$(git rev-parse --short HEAD)" >> $GITHUB_ENV

      - name: Azure Login
        uses: azure/login@v1
        with:
          creds: ${ secrets.AZURE_CREDENTIALS }

      - name: Login to ACR
        run: az acr login --name ${ secrets.ACR_NAME }

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '18'

      - name: Build Frontend Assets
        run: |
          cd frontend
          npm ci
          npm run build

      - name: Build & Push Backend Image
        run: |
          ACR=${ secrets.ACR_NAME }.azurecr.io
          docker build -t ${ACR}/qcms-backend:${ env.QCMS_TAG } -f backend/Dockerfile ./backend
          docker push ${ACR}/qcms-backend:${ env.QCMS_TAG }
          docker tag ${ACR}/qcms-backend:${ env.QCMS_TAG } ${ACR}/qcms-backend:latest
          docker push ${ACR}/qcms-backend:latest

      - name: Build & Push Frontend Image
        run: |
          ACR=${ secrets.ACR_NAME }.azurecr.io
          docker build -t ${ACR}/qcms-frontend:${ env.QCMS_TAG } -f frontend/Dockerfile ./frontend
          docker push ${ACR}/qcms-frontend:${ env.QCMS_TAG }
          docker tag ${ACR}/qcms-frontend:${ env.QCMS_TAG } ${ACR}/qcms-frontend:latest
          docker push ${ACR}/qcms-frontend:latest

      - name: Run Database Migrations
        run: |
          az containerapp job start \
            --name qcms-migration-job \
            --resource-group ${ env.RG }

      - name: Update Backend, Celery Worker & Beat
        run: |
          ACR=${ secrets.ACR_NAME }.azurecr.io
          for app in qcms-backend qcms-celery-worker qcms-celery-beat; do
            az containerapp update -n $app -g ${ env.RG } \
              --image ${ACR}/qcms-backend:${ env.QCMS_TAG }
          done

      - name: Update Frontend
        run: |
          ACR=${ secrets.ACR_NAME }.azurecr.io
          az containerapp update \

```

```
-n qcms-frontend \
-g ${env.RG} \
--image ${ACR}/qcms-frontend:${env.QCMS_TAG}
```

14.1 Required GitHub Secrets

Secret Name	Value
<code>AZURE_CREDENTIALS</code>	JSON output from <code>az ad sp create-for-rbac</code> (see below)
<code>ACR_NAME</code>	Your ACR registry name (without <code>.azurecr.io</code> suffix)

```
# Create Service Principal for GitHub Actions
az ad sp create-for-rbac \
  --name "qcms-github-actions" \
  --role contributor \
  --scopes /subscriptions/<SUBSCRIPTION_ID>/resourceGroups/$RG \
  --sdk-auth

# Copy the full JSON output and paste as AZURE_CREDENTIALS in GitHub Secrets
```

15. Post-Deployment Verification Checklist

Infrastructure

- ☐ PostgreSQL Flexible Server is running and accepting connections
- ☐ pgvector extension enabled: `SELECT * FROM pg_extension WHERE extname = 'vector';`
- ☐ Redis cache reachable from backend containers (check logs for connection success)
- ☐ Azure Blob Storage container `qcms-uploads` exists and is **PRIVATE** (no public access)
- ☐ Azure Key Vault populated with all secrets listed in Section 7
- ☐ All 4 Container Apps show **Running** status in Azure Portal
- ☐ ACR contains `qcms-backend:latest` and `qcms-frontend:latest` images

Application Health

- ☐ Frontend URL loads the QCMS login page without JavaScript console errors
- ☐ API health check: `curl https://<backend-url>/api/health` returns HTTP 200
- ☐ SuperAdmin login works successfully
- ☐ Super Admin portal (`/admin/super-admin.html`) loads without errors
- ☐ Create a test organization through the SuperAdmin portal
- ☐ Register a test user — verify invitation email is received via Resend
- ☐ Upload a test file — verify it appears in Azure Blob Storage
- ☐ Generate a test PDF QC Storybook report — verify download works
- ☐ Create a test project and advance through at least 2 stages
- ☐ Verify multilingual i18n toggle works (switch UI to Hindi)

Celery Workers

- ☐ Celery Worker logs show: `celery@<hostname> ... ready`
- ☐ Celery Beat logs show: `beat: Starting ...`
- ☐ Send a test notification — verify email arrives
- ☐ Exactly **1 replica** of Celery Beat is running (verify in Azure Portal)

Security

- [] Accessing backend URL directly from browser returns connection refused (internal ingress)
- [] Unauthenticated API calls return **HTTP 401 Unauthorized**
- [] Team Member account cannot access admin-only routes (returns HTTP 403)
- [] Password reset email flow works end-to-end
- [] SSL certificate is valid and HTTPS is enforced on custom domain
- [] **CHANGE SuperAdmin password immediately after first login!**

16. Scaling & Performance Tuning

16.1 Container App Scaling Rules

Service	Min Replicas	Max Replicas	Scale Trigger
Backend API	2	8	HTTP requests/sec > 100
Frontend (Nginx)	2	10	HTTP requests/sec > 200
Celery Worker	1	4	Redis queue depth > 10 tasks
Celery Beat	1	1	Fixed singleton — never scale

16.2 Gunicorn Configuration by Scale

Setting	Small (≤10 users)	Medium (10–100)	Large (100+)
<code>GUNICORN_WORKERS</code>	2	4	4 (scale via replicas)
<code>GUNICORN_THREADS</code>	2	4	8
<code>GUNICORN_TIMEOUT</code>	60s	60s	120s
Container Replicas	2	3–4	6–8

16.3 SQLAlchemy Connection Pool

```
# backend/app/config/database.py
SQLALCHEMY_ENGINE_OPTIONS = {
    'pool_size': 10,      # Persistent connections per worker
    'max_overflow': 20,   # Extra connections when pool is full
    'pool_timeout': 30,   # Seconds to wait for a pool slot
    'pool_recycle': 1800, # Recycle stale connections every 30 min
    'pool_pre_ping': True # Validate connection health before use
}
```

17. Backup & Disaster Recovery

17.1 PostgreSQL Backups

- Frequency:** Full daily backups + transaction log snapshots every 5 minutes
- Retention:** 7 days default (configurable up to 35 days)
- PITR:** Point-in-time restore available for any moment within the retention window


```
# Enable 14-day backup retention with geo-redundancy
az postgres flexible-server update \
  --resource-group $RG \
  --name $PG_SERVER \
  --backup-retention 14 \
  --geo-redundant-backup Enabled
```

17.2 RTO / RPO Objectives

Component	RTO	RPO
Backend API (ACA)	< 5 minutes (auto-restart)	0 — Stateless
Frontend Nginx (ACA)	< 3 minutes (auto-restart)	0 — Stateless
PostgreSQL Database	< 30 minutes (PITR or failover)	< 5 minutes
Redis Cache	< 10 minutes (Azure managed)	Rebuildable — cache data
Blob Storage (GRS)	< 1 hour (cross-region failover)	< 15 minutes

17.3 Point-in-Time Database Restore

```
az postgres flexible-server restore \
  --resource-group $RG \
  --name qcms-postgres-restored \
  --source-server qcms-postgres-server \
  --restore-time "2026-09-07T12:00:00Z"
```

18. Troubleshooting Reference

Problem	Possible Cause	Resolution
Backend returns 500 on startup	Missing env var or DB unreachable	<code>az containerapp logs show -n qcms-backend --follow</code> — look for <code>OperationalError</code> or <code>KeyError</code>
Login works but API calls return 401	<code>JWT_SECRET_KEY</code> mismatch between replicas	Verify all replicas use same Key Vault secret for <code>JWT-SECRET-KEY</code>
File uploads fail silently	<code>STORAGE_BACKEND</code> not <code>azure</code> or wrong connection string	Verify <code>STORAGE_BACKEND=azure</code> and <code>AZURE_STORAGE_CONNECTION_STRING</code> from Key Vault
Emails not delivered	Celery Worker not running or invalid Resend key	Check Celery Worker logs. Verify <code>RESEND_API_KEY</code> . Verify sender domain in Resend dashboard
RAG search returns no results	pgvector extension not enabled	<code>CREATE EXTENSION IF NOT EXISTS vector;</code> via psql
Celery Beat creates duplicate tasks	Multiple Celery Beat instances running	Ensure <code>--max-replicas 1</code> on Celery Beat container app
Frontend shows blank white screen	API URL wrong or Nginx can't reach backend	Check <code>nginx.conf</code> <code>proxy_pass</code> points to correct backend ACA internal FQDN
Migration job fails	Alembic conflict or missing DB permissions	Check job logs. Run <code>flask db current</code> to inspect migration state
Redis connection refused	Using <code>redis://</code> instead of <code>rediss://</code> for Azure TLS	Azure Redis requires <code>rediss://</code> on port <code>6380</code> — NOT <code>redis://</code> on <code>6379</code>
Containers not scaling up	Min replicas = 0 (cold start) or no scaling rules	Set <code>--min-replicas 1</code> to prevent cold starts. Define HTTP scaling rules
SuperAdmin cannot login	<code>SUPER_ADMIN_USERNAME</code> / <code>SUPER_ADMIN_PASSWORD</code> env vars missing	Verify both vars are set in backend container app environment
PDF generation fails	<code>fpdf2</code> dependency missing or storage write error	Verify <code>fpdf2 ≥ 2.7.5</code> in <code>requirements.txt</code> . Check blob storage write permissions

19. Cost Estimation

i Costs vary by Azure region and actual usage. Use the [Azure Pricing Calculator](#) for exact quotes.

19.1 Small Organization (up to 50 Users)

Service	Configuration	Estimated Monthly
Azure Container Apps	4 containers, ~2 replicas, Consumption plan	~\$40–\$80
Azure PostgreSQL Flexible	Standard_B2ms, 32 GB storage	~\$50–\$70
Azure Cache for Redis	C1 Standard (1 GB)	~\$50–\$55
Azure Blob Storage	Standard LRS, ~50 GB	~\$5–\$10
Azure Container Registry	Basic (10 GB)	~\$5
Azure Key Vault	Standard, < 10 secrets	~\$5
Azure Monitor / Log Analytics	~5 GB/month ingestion	~\$5–\$15
TOTAL		~\$160–\$240 / month

19.2 Enterprise Deployment (200+ Users)

Service	Configuration	Estimated Monthly
Azure Container Apps	Dedicated plan, 8+ replicas, autoscale	~\$150–\$300
Azure PostgreSQL Flexible	Standard_D4s_v3, HA enabled, Geo-redundant backup	~\$200–\$400
Azure Cache for Redis	C3 Standard (6 GB) or Premium P1	~\$150–\$300
Azure Blob Storage	Standard GRS, ~500 GB + CDN	~\$30–\$60
Azure Front Door + WAF	Standard tier with DDoS protection	~\$100–\$200
Others (ACR, KV, Monitor)	Standard configurations	~\$50–\$100
TOTAL		~\$680–\$1,360 / month

19.3 Cost Optimization Tips

- **Reserved Instances** — 1-year or 3-year commitment for PostgreSQL and Redis saves up to 63%
- **Auto-scale to 0** — Enable for dev/staging environments to eliminate idle costs overnight
- **Blob Lifecycle Policy** — Move files older than 90 days to Cool/Archive tier automatically
- **Azure Cost Alerts** — Set budget alerts at 80% and 100% of monthly spend threshold
- **ACR Purge Policy** — Auto-delete untagged Docker images older than 7 days

Appendix — Quick Reference Commands

A. Restart All Services

```
for app in qcms-backend qcms-celery-worker qcms-celery-beat qcms-frontend; do
  az containerapp revision restart \
    --resource-group qcms-production-rg \
    --name $app
  echo "Restarted: $app"
done
```

B. Deploy a New Image Version

```
NEW_TAG="v1.2.3"
ACR="

```

C. Scale Backend Replicas

```
az containerapp scale \
  --name qcms-backend \
  --resource-group qcms-production-rg \
  --min-replicas 3 \
  --max-replicas 8
```

D. View Celery Queue Status

```
az containerapp exec \
  --name qcms-celery-worker \
  --resource-group qcms-production-rg \
  --command "celery -A celery_worker.celery inspect active"
```

E. Check Running Migrations

```
az containerapp job execution list \
  --name qcms-migration-job \
  --resource-group qcms-production-rg \
  --output table
```

F. Force-Close All Active Sessions (Emergency)

```
# Via SuperAdmin portal: Admin → Users → Force Logout All
# OR via API:
curl -X POST https://qcms.yourdomain.com/api/super-admin/force-logout-all \
  -H "Authorization: Bearer <superadmin-jwt-token>"
```

G. PostgreSQL Point-in-Time Restore

```
az postgres flexible-server restore \
  --resource-group qcms-production-rg \
  --name qcms-postgres-restored \
  --source-server qcms-postgres-server \
  --restore-time "2026-09-07T12:00:00Z"
```

H. Check All Container App Statuses

```
az containerapp list \
  --resource-group qcms-production-rg \
  --query "[].{Name:name, Status:properties.runningStatus, Replicas:properties.template.scale.minReplicas}" \
  --output table
```

Prepared by: Deployment Management Team | IFQM QCMS | September 2026

Classification: Internal — Confidential | Do not distribute externally

Repository: github.com/IFQM-QCMS/imfq-QCMS

Prepared by: Deployment Management Team | IFQM QCMS | September 2026

Classification: Internal — Confidential | github.com/IFQM-QCMS/imfq-QCMS